

LIST

```
In [ ]: # Create a List
thislist = ["apple", "banana", "cherry"]
print(thislist)
```

```
In [ ]: # Lists allow duplicate values
thislist = ["apple", "banana", "cherry", "apple", "cherry"]
print(thislist)
```

```
In [ ]: #The number of items in the list
thislist = ["apple", "banana", "cherry"]
print(len(thislist))
```

```
In [ ]: # List items can be of any data type
list1 = ["apple", "banana", "cherry"]
list2 = [1, 5, 7, 9, 3]
list3 = [True, False, False]
```

```
In [ ]: # A list can contain different data types
list1 = ["abc", 34, True, 40, "male"]
```

```
In [ ]: # Datatype of list
mylist = ["apple", "banana", "cherry"]
print(type(mylist))
```

```
In [ ]: # Using the list(.) constructor to make a list
thislist = list(("apple", "banana", "cherry"))
print(thislist)
```

List Indexing

```
In [ ]: # List items are indexed and you can access them by referring to the index
# Print the second item of the list
thislist = ["apple", "banana", "cherry"]
print(thislist[1])
```

```
In [ ]: # Negative Indexing
#Print the last item of the list
thislist = ["apple", "banana", "cherry"]
print(thislist[-1])
```

```
In [ ]: # Range of Indexes  
#Return the third, fourth, and fifth item  
  
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[2:5])
```

```
In [ ]: # return the items from the beginning to, but NOT including, "kiwi"  
  
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[:4])
```

```
In [ ]: # return the items from "cherry" to the end  
  
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[2:])
```

```
In [ ]: #return the items from "orange" (-4) to, but NOT including "mango" (-1):  
  
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[-4:-1])
```

```
In [ ]: #Check if Item Exists  
#To determine if a specified item is present in a list use the in keyword:  
  
#Check if "apple" is present in the list:  
  
thislist = ["apple", "banana", "cherry"]  
if "apple" in thislist:  
    print("Yes, 'apple' is in the fruits list")
```

Change Item Values

```
In [ ]:   
  
#Change the second item  
  
thislist = ["apple", "banana", "cherry"]  
thislist[1] = "blackcurrant"  
print(thislist)
```

```
In [ ]: # Change a Range of Item Values  
  
# Change the values "banana" and "cherry" with the values "blackcurrant" and  
  
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]  
thislist[1:3] = ["blackcurrant", "watermelon"]  
print(thislist)
```

```
In [ ]: # Change the second value by replacing it with two new values:
```

```
thislist = ["apple", "banana", "cherry"]  
thislist[1:2] = ["blackcurrant", "watermelon"]  
print(thislist)
```

```
In [ ]: # Change the second and third value by replacing it with one value:
```

```
thislist = ["apple", "banana", "cherry"]  
thislist[1:3] = ["watermelon"]  
print(thislist)
```

```
In [ ]: # Insert Items
```

```
# To insert a new List item, without replacing any of the existing values, use the insert()
```

```
# Insert "watermelon" as the third item
```

```
thislist = ["apple", "banana", "cherry"]  
thislist.insert(2, "watermelon")  
print(thislist)
```

Add List Items

```
In [ ]: # Append Items
```

```
# Using the append() method to append an item
```

```
thislist = ["apple", "banana", "cherry"]  
thislist.append("orange")  
print(thislist)
```

```
In [ ]: # Insert Items
```

```
#Insert an item as the second position
```

```
thislist = ["apple", "banana", "cherry"]  
thislist.insert(1, "orange")  
print(thislist)
```

```
In [ ]: # Extend List
```

```
# Add the elements of tropical to thislist
```

```
thislist = ["apple", "banana", "cherry"]  
tropical = ["mango", "pineapple", "papaya"]  
thislist.extend(tropical)  
print(thislist)
```

```
In [ ]: # Add Any Iterable  
# The extend() method does not have to append lists, you can add any iterable  
  
# Add elements of a tuple to a list  
  
thislist = ["apple", "banana", "cherry"]  
thistuple = ("kiwi", "orange")  
thislist.extend(thistuple)  
print(thislist)
```

Remove Specified Item

```
In [ ]:   
  
# Remove "banana"  
  
thislist = ["apple", "banana", "cherry"]  
thislist.remove("banana")  
print(thislist)
```

```
In [ ]: # Remove the first occurrence of "banana"  
  
thislist = ["apple", "banana", "cherry", "banana", "kiwi"]  
thislist.remove("banana")  
print(thislist)
```

```
In [ ]: # Remove Specified Index  
# The pop() method removes the specified index. If you do not specify the index, the method will remove the last item  
  
# Remove the second item  
  
thislist = ["apple", "banana", "cherry"]  
thislist.pop(1)  
print(thislist)
```

```
In [ ]: # Remove the Last item  
  
thislist = ["apple", "banana", "cherry"]  
thislist.pop()  
print(thislist)
```

In []: *# The del keyword also removes the specified index*

Remove the first item

```
thislist = ["apple", "banana", "cherry"]  
del thislist[0]  
print(thislist)
```

In []: *# The del keyword can also delete the list completely.*

Delete the entire list

```
thislist = ["apple", "banana", "cherry"]  
del thislist
```

In []: *# Clear the List*

The clear() method empties the list. The list still remains, but it has no content

Clear the list content

```
thislist = ["apple", "banana", "cherry"]  
thislist.clear()  
print(thislist)
```

Loop Through a List

In []:

Print all items in the list, one by one:

```
thislist = ["apple", "banana", "cherry"]  
for x in thislist:  
    print(x)
```

In []: *# Loop Through the Index Numbers*

You can also loop through the list items by referring to their index number

Print all items by referring to their index number

```
thislist = ["apple", "banana", "cherry"]  
for i in range(len(thislist)):  
    print(thislist[i])
```



```
In [ ]: # Using a While Loop  
# You can loop through the list items by using a while loop.  
  
# Print all items, using a while loop to go through all the index numbers  
  
thislist = ["apple", "banana", "cherry"]  
i = 0  
while i < len(thislist):  
    print(thislist[i])  
    i = i + 1
```

List Comprehension

```
In [ ]: # List comprehension offers a shorter syntax when you want to create a new  
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
newlist = []  
  
for x in fruits:  
    if "a" in x:  
        newlist.append(x)  
  
print(newlist)
```

```
In [ ]: fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
  
newlist = [x for x in fruits if "a" in x]  
  
print(newlist)
```

```
In [ ]: # The Syntax  
    # newlist = [expression for item in iterable if condition == True]  
    # The return value is a new list, leaving the old list unchanged.  
  
#Condition  
#The condition is like a filter that only accepts the items that evaluate to  
  
# Only accept items that are not "apple":  
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
newlist = [x for x in fruits if x != "apple"]  
print(newlist)  
  
# With no if statement:  
  
newlist = [x for x in fruits]  
print(newlist)
```

```
In [ ]: # Iterable
# The iterable can be any iterable object, like a list, tuple, set etc.

# You can use the range() function to create an iterable:
newlist = [x for x in range(10)]
print(newlist)

#Accept only numbers lower than 5:

newlist = [x for x in range(10) if x < 5]
print(newlist)
```

```
In [ ]: # Expression
# The expression is the current item in the iteration, but it is also the o

# Set the values in the new list to upper case:

newlist = [x.upper() for x in fruits]
print(newlist)

#Set all values in the new list to 'hello':

newlist = ['hello' for x in fruits]
print(newlist)

# Return "orange" instead of "banana":

newlist = [x if x != "banana" else "orange" for x in fruits]
print(newlist)
```

Sort List Alphanumerically

```
In [ ]: # List objects have a sort() method that will sort the list alphanumerically

#Sort the list alphabetically:

thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort()
print(thislist)
```

```
In [ ]: # Sort the list numerically:

thislist = [100, 50, 65, 82, 23]
thislist.sort()
print(thislist)
```

```
In [ ]: # Sort Descending
# To sort descending, use the keyword argument reverse = True

# Sort the List descending:

thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort(reverse = True)
print(thislist)
```

```
In [ ]: # Sort the List descending:

thislist = [100, 50, 65, 82, 23]
thislist.sort(reverse = True)
print(thislist)
```

```
In [ ]: # Customize Sort Function
# You can also customize your own function by using the keyword argument key

# Sort the List based on how close the number is to 50

def myfunc(n):
    return abs(n - 50)

thislist = [100, 50, 65, 82, 23]
thislist.sort(key = myfunc)
print(thislist)
```

```
In [ ]: # Case Insensitive Sort
# By default the sort() method is case sensitive, resulting in all capital

# Case sensitive sorting can give an unexpected result

thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.sort()
print(thislist)
```

```
In [ ]: # Perform a case-insensitive sort of the list:

thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.sort(key = str.lower)
print(thislist)
```



```
In [ ]: # Reverse Order  
#The reverse() method reverses the current sorting order of the elements.  
  
# Reverse the order of the List items  
  
thislist = ["banana", "Orange", "Kiwi", "cherry"]  
thislist.reverse()  
print(thislist)
```

Copy a List

```
In [ ]:   
  
# You can use the built-in List method copy() to copy a List.  
  
# Make a copy of a List with the copy() method:  
  
thislist = ["apple", "banana", "cherry"]  
mylist = thislist.copy()  
print(mylist)
```

```
In [ ]: #Use the list() method  
# Another way to make a copy is to use the built-in method list().  
  
# Make a copy of a List with the list() method:  
  
thislist = ["apple", "banana", "cherry"]  
mylist = list(thislist)  
print(mylist)
```

```
In [ ]: # Use the slice Operator  
# You can also make a copy of a List by using the : (slice) operator.  
  
# Make a copy of a List with the : operator:  
  
thislist = ["apple", "banana", "cherry"]  
mylist = thislist[:]  
print(mylist)
```

Join Two Lists

In []:

```
# There are several ways to join, or concatenate, two or more Lists in Python

# Join two List:

list1 = ["a", "b", "c"]
list2 = [1, 2, 3]

list3 = list1 + list2
print(list3)
```

In []:

```
# Another way to join two Lists is by appending all the items from list2 in

# Append list2 into list1:

list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]

for x in list2:
    list1.append(x)

print(list1)
```

In []:

```
# Use the extend() method to add list2 at the end of list1:

list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]

list1.extend(list2)
print(list1)
```

List Methods

Python has a set of built-in methods that you can use on lists.

append() - Adds an element at the end of the list

clear() - Removes all the elements from the list

copy() - Returns a copy of the list

count() - Returns the number of elements with the specified value

extend() - Add the elements of a list (or any iterable), to the end of the current list

index() - Returns the index of the first element with the specified

In []: